

Title: Capture and Analyze Network Traffic Using Wireshark.

- Name: VIPIN KUMAR
- Training name: CYBER SECURITY INTERNSHIP
- Task5: Capture and Analyze Network Traffic Using Wireshark.
- Date: 30-07-2026

Objective:

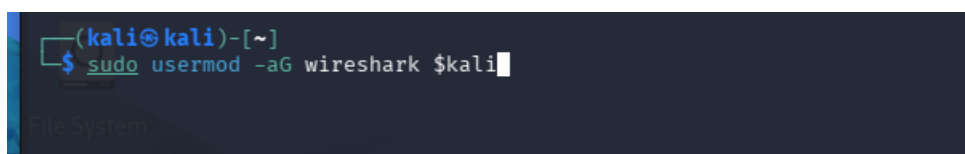
To capture live network traffic using **Wireshark**, analyze the captured packets, and identify different network protocols such as **DNS, TCP, and ICMP**. The task aims to develop practical skills in packet capture, protocol filtering, packet analysis, and understanding basic network communication.

1. Install Wireshark



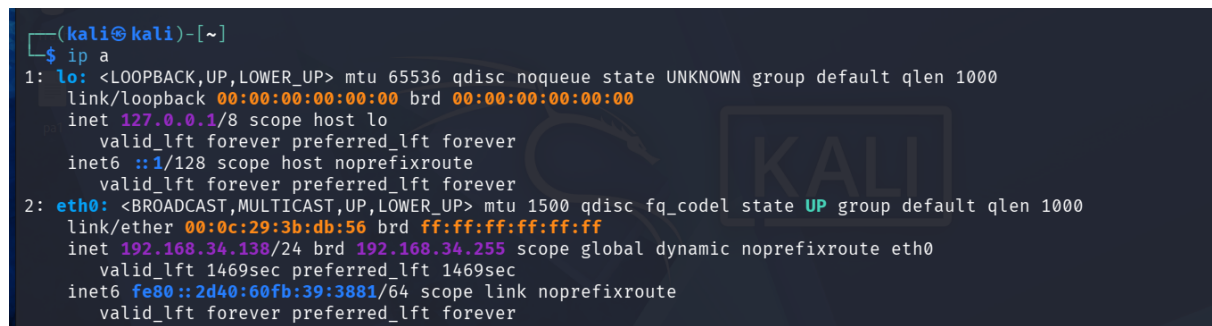
```
(kali@kali)-[~]
$ sudo apt update
sudo apt install wireshark -y
```

During install, if prompted "Should non-superusers be able to capture packets?" → select Yes



```
(kali@kali)-[~]
$ sudo usermod -aG wireshark $kali
```

2. Identify your active network interface



```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:3b:db:56 brd ff:ff:ff:ff:ff:ff
   inet 192.168.34.138/24 brd 192.168.34.255 scope global dynamic noprefixroute eth0
       valid_lft 1469sec preferred_lft 1469sec
   inet6 fe80::2d40:60fb:39:3881/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

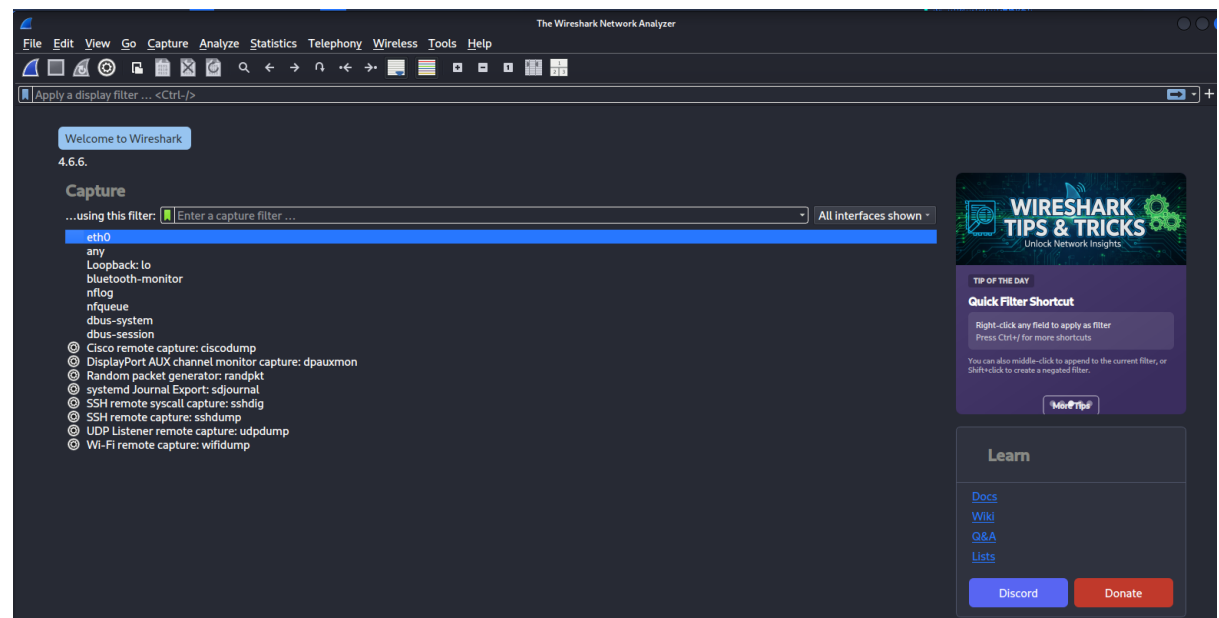
Look for the interface that has an IP address (e.g., eth0, wlan0, enp0s3). You'll pick this one in Wireshark.

3. Launch Wireshark

```
(kali㉿kali)-[~]  
$ wireshark  
** (wireshark:5562) 12:34:25.057991 [GUI WARNING] -- Failed to register with host portal QDBusError("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")
```

4. Start capturing

- In the Wireshark start screen, you'll see a list of interfaces with live traffic graphs.
- Double-click your active interface (e.g., eth0 or wlan0) to start capturing immediately.



5. Generate traffic while capturing

Open a terminal alongside Wireshark and generate some real traffic:

eg: ping -c 5 google.com

You can also do:

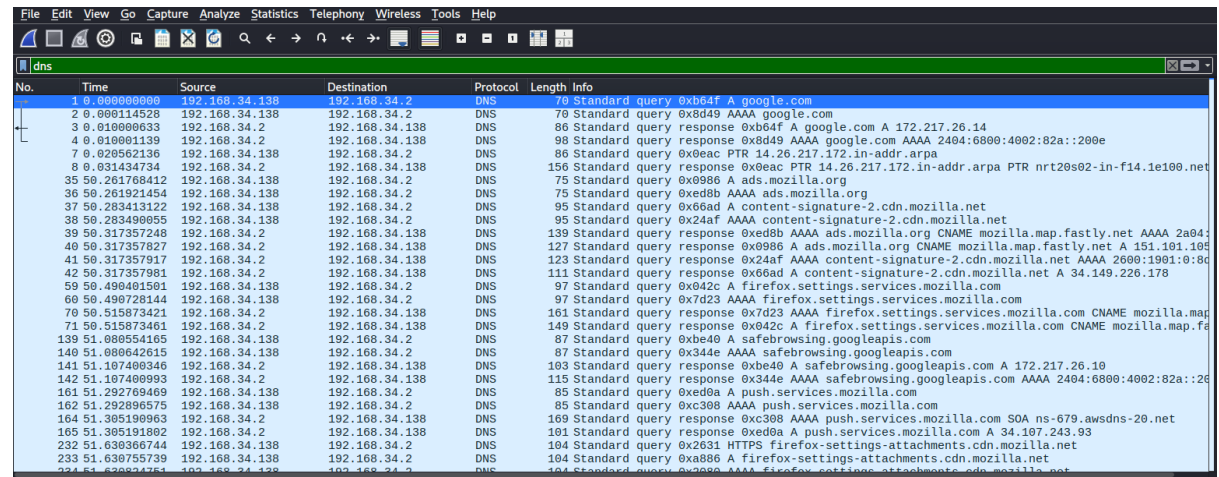
nslookup google.com

curl <http://example.com>

6. Stop the capture after ~5 minute

Click the red **Stop** button (square icon) in Wireshark's toolbar.

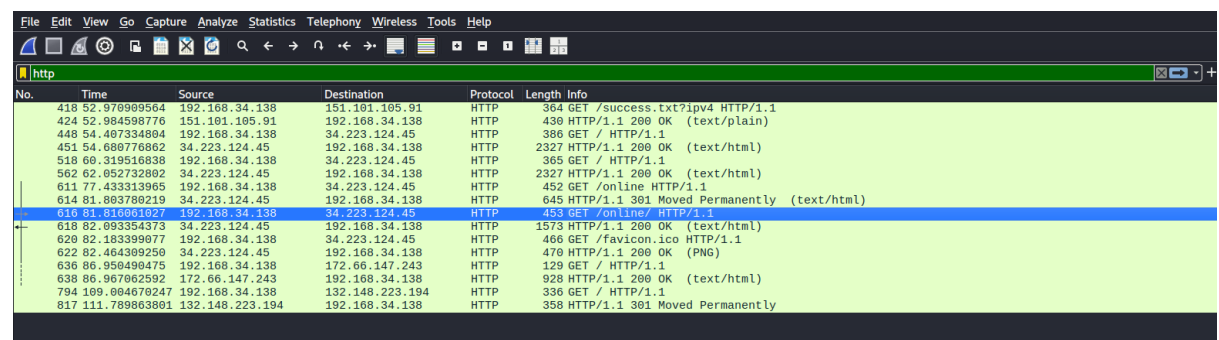
DNS:



The screenshot shows the Wireshark interface with the DNS filter applied. The packet list pane displays 104 captured DNS packets. The first packet is a standard query for google.com. Subsequent packets show responses and other queries for various domains including mozilla.org, mozilla-map.fastly.net, and push.services.mozilla.com. The packet details pane on the right shows the structure of a DNS Standard query response.

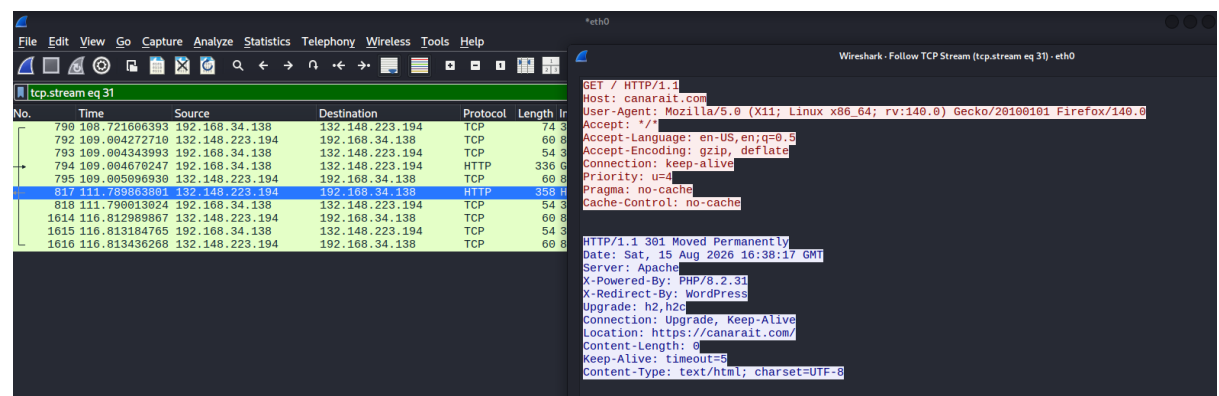
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.34.138	192.168.34.2	DNS	70	Standard query 0xb64f A google.com
2	0.000114528	192.168.34.138	192.168.34.2	DNS	70	Standard query 0xb649 AAAA google.com
3	0.010000033	192.168.34.2	192.168.34.138	DNS	86	Standard query response 0xb64f A google.com A 172.217.26.14
4	0.010001139	192.168.34.2	192.168.34.138	DNS	98	Standard query response 0xb649 AAAA google.com AAAA 2404:6800:4002:82a::200e
7	0.020562136	192.168.34.138	192.168.34.2	DNS	86	Standard query 0x0eac PTR 14.26.217.172.in-addr.arpa
8	0.031434734	192.168.34.2	192.168.34.138	DNS	156	Standard query response 0x0eac PTR 14.26.217.172.in-addr.arpa PTR nrt20s02-in-f14.1e100.net
35	50.261768412	192.168.34.138	192.168.34.2	DNS	75	Standard query 0x0986 A ads.mozilla.org
36	50.261921454	192.168.34.138	192.168.34.2	DNS	75	Standard query 0xed8b AAAA ads.mozilla.org
37	50.283413122	192.168.34.138	192.168.34.2	DNS	95	Standard query 0xb6ad A content-signature-2.cdn.mozilla.net
38	50.283490055	192.168.34.138	192.168.34.2	DNS	95	Standard query 0x24af AAAA content-signature-2.cdn.mozilla.net
39	50.317357248	192.168.34.2	192.168.34.138	DNS	139	Standard query response 0xed8b AAAA ads.mozilla.org CNAME mozilla.map.fastly.net AAAA 2a04:...
40	50.317357827	192.168.34.2	192.168.34.138	DNS	127	Standard query response 0x0986 A ads.mozilla.org CNAME mozilla.map.fastly.net A 151.101.105...
41	50.317357917	192.168.34.2	192.168.34.138	DNS	123	Standard query response 0x24af AAAA content-signature-2.cdn.mozilla.net AAAA 2600:1901:0:8d...
42	50.317357981	192.168.34.2	192.168.34.138	DNS	111	Standard query response 0xb6ad A content-signature-2.cdn.mozilla.net A 34.149.226.178
50	50.490401501	192.168.34.138	192.168.34.2	DNS	97	Standard query 0xb42c A firefox.settings.services.mozilla.com
60	50.490728144	192.168.34.138	192.168.34.2	DNS	97	Standard query 0x7d23 AAAA firefox.settings.services.mozilla.com
70	50.515873421	192.168.34.2	192.168.34.138	DNS	161	Standard query response 0x7d23 AAAA firefox.settings.services.mozilla.com CNAME mozilla.map...
71	50.515873461	192.168.34.2	192.168.34.138	DNS	149	Standard query response 0x042c A firefox.settings.services.mozilla.com CNAME mozilla.map.f...
139	51.080554165	192.168.34.138	192.168.34.2	DNS	87	Standard query 0xb640 A safebrowsing.googleapis.com
140	51.080642615	192.168.34.138	192.168.34.2	DNS	87	Standard query 0x344e AAAA safebrowsing.googleapis.com
141	51.107406346	192.168.34.2	192.168.34.138	DNS	103	Standard query response 0xb640 A safebrowsing.googleapis.com A 172.217.26.10
142	51.107406993	192.168.34.2	192.168.34.138	DNS	115	Standard query response 0x344e AAAA safebrowsing.googleapis.com AAAA 2404:6800:4002:82a::26...
161	51.292769469	192.168.34.138	192.168.34.2	DNS	85	Standard query 0xed0a A push.services.mozilla.com
162	51.292896575	192.168.34.138	192.168.34.2	DNS	85	Standard query 0xc308 AAAA push.services.mozilla.com
164	51.305190963	192.168.34.2	192.168.34.138	DNS	169	Standard query response 0xc308 AAAA push.services.mozilla.com SOA ns-679.awsdns-20.net
165	51.305191082	192.168.34.138	192.168.34.138	DNS	161	Standard query response 0xed0a A push.services.mozilla.com A 34.107.243.93
232	51.630366744	192.168.34.138	192.168.34.2	DNS	104	Standard query 0x2631 HTTPS firefox-settings-attachments.cdn.mozilla.net
233	51.630755739	192.168.34.138	192.168.34.2	DNS	104	Standard query 0xa886 A firefox-settings-attachments.cdn.mozilla.net
234	51.630824754	192.168.34.138	192.168.34.2	DNS	104	Standard query 0x2000 AAAA firefox-settings-attachments.cdn.mozilla.net

Http:



The screenshot shows the Wireshark interface with the HTTP filter applied. The packet list pane displays 18 captured HTTP packets. The first packet is a GET request for success.txt?ipv4. Subsequent packets show responses and other requests for various resources including favicon.ico and a 301 Moved Permanently response. The packet details pane on the right shows the structure of an HTTP GET request.

No.	Time	Source	Destination	Protocol	Length	Info
418	52.970095564	192.168.34.138	151.101.105.91	HTTP	364	GET /success.txt?ipv4 HTTP/1.1
424	52.984598776	151.101.105.91	192.168.34.138	HTTP	430	HTTP/1.1 200 OK (text/plain)
448	54.407334804	192.168.34.138	34.223.124.45	HTTP	386	GET / HTTP/1.1
451	54.600776862	34.223.124.45	192.168.34.138	HTTP	2327	HTTP/1.1 200 OK (text/html)
518	60.319516038	192.168.34.138	34.223.124.45	HTTP	365	GET / HTTP/1.1
562	62.052732802	34.223.124.45	192.168.34.138	HTTP	2327	HTTP/1.1 200 OK (text/html)
611	77.433313965	192.168.34.138	34.223.124.45	HTTP	452	GET /online/ HTTP/1.1
614	81.803780219	34.223.124.45	192.168.34.138	HTTP	645	HTTP/1.1 301 Moved Permanently (text/html)
616	81.81001027	192.168.34.138	34.223.124.45	HTTP	453	GET /online/ HTTP/1.1
618	82.093354373	34.223.124.45	192.168.34.138	HTTP	1573	HTTP/1.1 200 OK (text/html)
620	82.183399077	192.168.34.138	34.223.124.45	HTTP	466	GET /favicon.ico HTTP/1.1
622	82.464309250	34.223.124.45	192.168.34.138	HTTP	479	HTTP/1.1 200 OK (PNG)
636	86.950490475	192.168.34.138	172.66.147.243	HTTP	129	GET / HTTP/1.1
638	86.967062592	172.66.147.243	192.168.34.138	HTTP	928	HTTP/1.1 200 OK (text/html)
794	109.004670247	192.168.34.138	132.148.223.194	HTTP	336	GET / HTTP/1.1
817	111.789863801	132.148.223.194	192.168.34.138	HTTP	358	HTTP/1.1 301 Moved Permanently



The screenshot shows the Wireshark interface with the TCP stream filter applied. The packet list pane displays 10 captured TCP packets. The packet details pane on the right shows the structure of a TCP stream, including the GET request and the 301 Moved Permanently response. The packet bytes pane on the right shows the raw data of the GET request and the 301 Moved Permanently response.

No.	Time	Source	Destination	Protocol	Length	Info
790	100.721606393	192.168.34.138	132.148.223.194	TCP	74	80 → 80 [RST] Seq=100721606393 Win=0 Len=0
792	109.004272710	132.148.223.194	192.168.34.138	TCP	60	80 → 80 [RST] Seq=100721606393 Win=0 Len=0
793	109.004343993	192.168.34.138	132.148.223.194	TCP	54	80 → 80 [RST] Seq=100721606393 Win=0 Len=0
794	109.004670247	192.168.34.138	132.148.223.194	HTTP	336	GET / HTTP/1.1
795	109.005090930	132.148.223.194	192.168.34.138	TCP	60	80 → 80 [RST] Seq=100721606393 Win=0 Len=0
817	111.789863801	132.148.223.194	192.168.34.138	HTTP	358	HTTP/1.1 301 Moved Permanently
818	111.799013024	192.168.34.138	132.148.223.194	TCP	54	80 → 80 [RST] Seq=100721606393 Win=0 Len=0
1614	116.812989867	132.148.223.194	192.168.34.138	TCP	60	80 → 80 [RST] Seq=100721606393 Win=0 Len=0
1615	116.813184765	192.168.34.138	132.148.223.194	TCP	54	80 → 80 [RST] Seq=100721606393 Win=0 Len=0
1616	116.813436268	132.148.223.194	192.168.34.138	TCP	60	80 → 80 [RST] Seq=100721606393 Win=0 Len=0

GET / HTTP/1.1
Host: canarait.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Priority: u=0
Pragma: no-cache
Cache-Control: no-cache

HTTP/1.1 301 Moved Permanently
Date: Sat, 15 Aug 2020 16:38:17 GMT
Server: Apache
X-Powered-By: PHP/8.2.31
X-Redirect-By: WordPress
Upgrade: h2,h2c
Connection: Upgrade, Keep-Alive
Location: https://canarait.com/
Content-Length: 0
Keep-Alive: timeout=5
Content-Type: text/html; charset=UTF-8

Tcp:

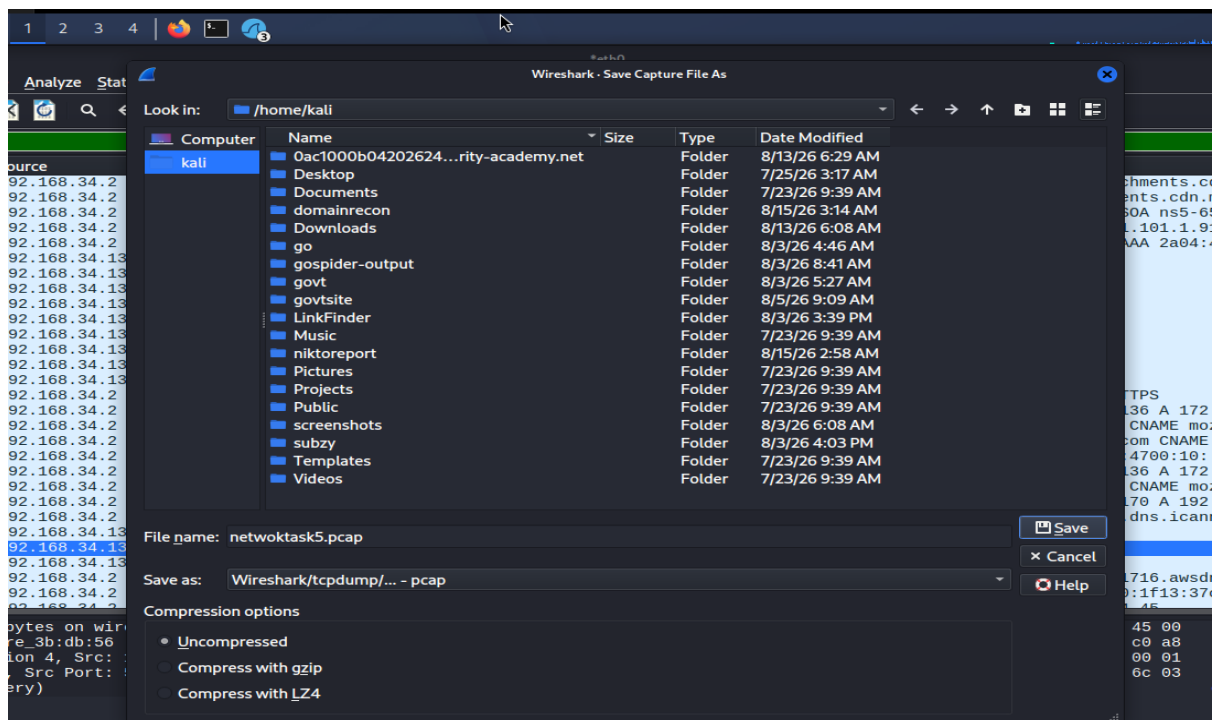
No.	Time	Source	Destination	Protocol	Length	Info
565	62.054253482	34.223.124.45	192.168.34.138	TCP	60	443 → 57770 [ACK] Seq=4514 Ack=1292 Win=64240 Len=0
566	62.054289420	192.168.34.138	34.223.124.45	TCP	54	57770 → 443 [FIN, ACK] Seq=1292 Ack=4514 Win=65535 Len=0
567	62.054504189	34.223.124.45	192.168.34.138	TCP	60	443 → 57770 [ACK] Seq=4514 Ack=1293 Win=64239 Len=0
572	62.082360552	192.168.34.138	34.223.124.45	TCP	74	58594 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=967070288 TSecr=0 WS=256
573	62.138490637	34.223.124.45	192.168.34.138	TLSv1.3	233	Application Data
574	62.180800528	192.168.34.138	34.223.124.45	TCP	54	57782 → 443 [ACK] Seq=758 Ack=4514 Win=65535 Len=0
575	62.333517077	192.168.34.138	34.223.124.45	TCP	74	58606 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=681122835 TSecr=0 WS=256
576	63.093176392	192.168.34.138	34.223.124.45	TCP	74	[TCP Retransmission] 58594 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=967070288 TSecr=0 WS=256
577	63.340781841	192.168.34.138	34.223.124.45	TCP	74	[TCP Retransmission] 58606 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=681122835 TSecr=0 WS=256
578	64.115749193	192.168.34.138	34.223.124.45	TCP	74	[TCP Retransmission] 58594 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=967070288 TSecr=0 WS=256
579	64.327243136	192.168.34.138	34.223.124.45	TCP	74	[TCP Retransmission] 58606 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=681122835 TSecr=0 WS=256
580	64.628673101	192.168.34.138	34.223.124.45	TCP	74	[TCP Retransmission] 56122 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=967070288 TSecr=0 WS=256
581	65.14068325	192.168.34.138	34.223.124.45	TCP	74	[TCP Retransmission] 58594 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=967070288 TSecr=0 WS=256
582	65.398832005	192.168.34.138	34.223.124.45	TCP	74	[TCP Retransmission] 58606 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=681122835 TSecr=0 WS=256
583	66.040489464	192.168.34.138	34.223.124.45	TCP	54	58590 → 80 [FIN, ACK] Seq=1 Ack=1 Win=64240 Len=0
584	66.041057837	34.223.124.45	192.168.34.138	TCP	60	80 → 58590 [ACK] Seq=1 Ack=2 Win=64239 Len=0
585	66.041063170	192.168.34.138	34.223.124.45	TCP	74	[TCP Retransmission] 58594 → 80 [FIN, ACK] Seq=1 Ack=2 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=967070288 TSecr=0 WS=256
586	66.342052413	34.223.124.45	192.168.34.138	TCP	60	80 → 58590 [FIN, PSH, ACK] Seq=1 Ack=2 Win=64239 Len=0
587	66.342119663	192.168.34.138	34.223.124.45	TCP	54	58590 → 80 [ACK] Seq=2 Ack=2 Win=64240 Len=0
588	66.420715713	192.168.34.138	34.223.124.45	TCP	74	[TCP Retransmission] 58606 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=681122835 TSecr=0 WS=256
589	66.420715713	192.168.34.138	34.223.124.45	TLSv1.3	78	Application Data
590	66.878495909	192.168.34.138	34.223.124.45	TCP	54	57770 → 443 [RST] Seq=1293 Win=0 Len=0
591	67.058320978	192.168.34.138	34.223.124.45	TCP	54	58588 → 80 [FIN, ACK] Seq=312 Ack=2274 Win=65535 Len=0
592	67.059183444	34.223.124.45	192.168.34.138	TCP	60	80 → 58588 [ACK] Seq=2274 Ack=313 Win=64239 Len=0
593	67.059247704	192.168.34.138	34.223.124.45	TLSv1.3	78	Application Data
594	67.059381102	192.168.34.138	34.223.124.45	TCP	54	57782 → 443 [FIN, ACK] Seq=782 Ack=4514 Win=65535 Len=0
595	67.059604375	34.223.124.45	192.168.34.138	TCP	60	443 → 57782 [ACK] Seq=4514 Ack=782 Win=64240 Len=0
596	67.059806332	34.223.124.45	192.168.34.138	TCP	60	443 → 57782 [ACK] Seq=4514 Ack=783 Win=64239 Len=0

Icmp:

No.	Time	Source	Destination	Protocol	Length	Info
5	0.011055095	192.168.34.138	172.217.26.14	ICMP	98	Echo (ping) request id=0xb651, seq=1/256, ttl=64 (reply in 6)
6	0.020319148	172.217.26.14	192.168.34.138	ICMP	90	Echo (ping) reply id=0xb651, seq=1/256, ttl=128 (reply in 5)
9	1.012775880	192.168.34.138	172.217.26.14	ICMP	98	Echo (ping) request id=0xb651, seq=2/512, ttl=64 (reply in 10)
10	1.022735402	172.217.26.14	192.168.34.138	ICMP	98	Echo (ping) reply id=0xb651, seq=2/512, ttl=128 (request in 9)
11	2.014010893	192.168.34.138	172.217.26.14	ICMP	98	Echo (ping) request id=0xb651, seq=3/768, ttl=64 (reply in 12)
12	2.024071649	172.217.26.14	192.168.34.138	ICMP	98	Echo (ping) reply id=0xb651, seq=3/768, ttl=128 (request in 11)
13	3.015304625	192.168.34.138	172.217.26.14	ICMP	90	Echo (ping) request id=0xb651, seq=4/1024, ttl=64 (reply in 14)
14	3.055876298	172.217.26.14	192.168.34.138	ICMP	98	Echo (ping) reply id=0xb651, seq=4/1024, ttl=128 (request in 13)
15	4.016899647	192.168.34.138	172.217.26.14	ICMP	98	Echo (ping) request id=0xb651, seq=5/1280, ttl=64 (reply in 16)
16	4.040463005	172.217.26.14	192.168.34.138	ICMP	98	Echo (ping) reply id=0xb651, seq=5/1280, ttl=128 (request in 15)

9. Export the capture as .pcap

Go to **File** → **Save As**, choose a location, and save as capture.pcap (or .pcapng,



Conclusion:

In this task, Wireshark was used to capture and analyze live network traffic. The captured packets were examined using protocol-based filters to identify different types of network communication, including **DNS, TCP, and ICMP**. Packet details such as source and destination addresses, protocols, ports, and packet information were analyzed to understand how data travels across a network. The capture was then saved as a **.pcap file** for further analysis and documentation. Overall, the task provided practical experience in network packet capture, protocol identification, and basic traffic analysis